

Atividade de Revisão

P.O.O./Java — Aulas 01 e 02 (para estudo antes da prova)

Escola: _____	
Disciplina: Programação Orientada a Objetos (Java)	Professor: Alan Mateus
Aluno(a): _____ _____	Turma: _____
Data: ____ / ____ / _____	Atividade sem valor de nota — uso para estudo

Instruções:

- Leia com atenção todos os enunciados antes de responder.
- Todas as questões são objetivas (múltipla escolha, verdadeiro/falso, associação, completar ou ordenação) — não há questões abertas.
- Utilize caneta azul ou preta e assinale apenas uma alternativa em cada questão de múltipla escolha.
- Rasuras excessivas podem invalidar a resposta.
- Esta atividade não vale nota — use-a para revisar antes da prova.
- Algumas questões apresentam trechos de código Java (e uma comparação com C#) para interpretar.

Avaliação alinhada às Competências Gerais da BNCC, com destaque para a Competência 2 (pensamento científico, crítico e criativo) e a Competência 5 (cultura digital), aplicadas ao componente de Programação Orientada a Objetos do itinerário técnico.

Questões (Banco completo)

Aula 01 — Introdução à POO: Objetos e Classes

Questão 1 (Aula 1 — Múltipla escolha)

O que é Programação Orientada a Objetos (POO)?

- (A) Um paradigma de programação que organiza o código em objetos, combinando dados (atributos) e comportamentos (métodos).
- (B) Uma linguagem de programação específica, como Python ou Java.
- (C) Um tipo de banco de dados relacional.
- (D) Uma ferramenta de design gráfico para interfaces.
- (E) Um sistema operacional voltado para servidores.

Questão 2 (Aula 1 — Associação de colunas)

Associe a coluna A à coluna B, escrevendo o número correspondente entre parênteses:

Coluna A — Conceito	Coluna B — Analogia do mundo real
1. Classe	() Ações que a casa permite (abrir porta)
2. Objeto	() A casa construída a partir do projeto
3. Atributo	() Projeto/planta de uma casa
4. Método	() Características da casa (cor, tamanho)

Questão 3 (Aula 1 — Verdadeiro ou Falso)

Sobre classe e objeto, avalie as afirmações:

- () I. Uma classe é o molde que define atributos e métodos.
- () II. Um objeto é uma instância criada a partir de uma classe.
- () III. Atributos são funções que o objeto pode executar.

- () IV. Em Java, "new NomeDaClasse()" é usado para criar um objeto (instância).
- () V. Uma classe só pode gerar um único objeto durante todo o programa.

Questão 4 (Aula 1 — Múltipla escolha)

Considere a classe Aluno (com atributos nome, idade, nota e método apresentar()) e o trecho abaixo, dentro do main:

```
Aluno aluno1 = new Aluno();
aluno1.nome = "Maria";
aluno1.idade = 17;
aluno1.nota = 8.5;
aluno1.apresentar();
```

O que esse trecho faz, em ordem?

- (A) Declara a classe Aluno, sem criar nenhum objeto.
- (B) Cria um objeto da classe Aluno, atribui valores aos seus atributos e chama o método apresentar().
- (C) Cria três objetos diferentes da classe Aluno.
- (D) Apenas imprime a palavra "Aluno" na tela.
- (E) Gera um erro de compilação, pois "new" não existe em Java.

Questão 5 (Aula 1 — Múltipla escolha)

Na classe Aluno, o que são nome, idade e nota?

- (A) Métodos da classe
- (B) Objetos independentes
- (C) Atributos da classe
- (D) Classes separadas
- (E) Comentários do código

Questão 6 (Aula 1 — Múltipla escolha)

O que é o método apresentar() na classe Aluno?

- (A) Um atributo que guarda o nome do aluno.
- (B) Um erro de sintaxe.
- (C) Uma nova classe dentro de Aluno.
- (D) Uma função que o objeto pode executar, nesse caso para exibir seus dados.
- (E) Um comentário no código.

Questão 7 (Aula 1 — Análise de afirmações)

- I. Em Java, os atributos são declarados como "String nome;", enquanto em C# são declarados como "public string Nome;".
- II. A forma de criar um objeto ("new Aluno()") é a mesma tanto em Java quanto em C#.
- III. Em Java, o método usa "System.out.println", enquanto em C# usa "Console.WriteLine".

Está(ão) correta(s):

- (A) Nenhuma das afirmações
- (B) Apenas I e II
- (C) Apenas III
- (D) Apenas II
- (E) I, II e III

Questão 8 (Aula 1 — Complete com o banco de palavras)

Banco de palavras: classe | atributos | métodos | objeto

"Uma _____ é o molde que define _____ e _____; um _____ é a instância criada a partir dela."

Aula 02 — Atributos, Métodos e Construtores

Questão 9 (Aula 2 — Múltipla escolha)

O que é um construtor em uma classe Java?

- (A) Um método especial, chamado automaticamente ao criar um objeto com new, usado para inicializar os atributos.
- (B) Um atributo que armazena o nome da classe.
- (C) Um tipo de variável que só existe dentro de métodos.
- (D) Um comentário especial no início da classe.
- (E) Um método que só pode ser chamado manualmente, nunca automaticamente.

Questão 10 (Aula 2 — Múltipla escolha)

Quais características um construtor deve ter?

- (A) Nome diferente da classe e tipo de retorno void.
- (B) Mesmo nome da classe e nenhum tipo de retorno.
- (C) Nome qualquer e tipo de retorno obrigatório int.
- (D) Deve sempre receber pelo menos um parâmetro.
- (E) Deve ser sempre declarado como private.

Questão 11 (Aula 2 — Verdadeiro ou Falso)

Sobre construtores, avalie as afirmações:

- () I. Um construtor é chamado automaticamente quando um objeto é criado com a palavra-chave new.
- () II. Um construtor pode ter o mesmo nome de qualquer método da classe, não apenas da classe em si.
- () III. Uma classe pode ter mais de um construtor, desde que as assinaturas sejam diferentes.
- () IV. O construtor padrão (sem parâmetros) é obrigatório e deve sempre existir explicitamente no código.
- () V. Um construtor serve para inicializar os atributos de um objeto no momento de sua criação.

Questão 12 (Aula 2 — Múltipla escolha)

Considere o construtor abaixo:

```
public Aluno() {  
    nome = "Sem nome";  
    idade = 0;  
    nota = 0.0;  
}
```

Ao executar "Aluno a1 = new Aluno();" seguido de "a1.exibir();", qual será a saída exibida?

- (A) Maria | 17 | 8.5
- (B) null | 0 | 0.0
- (C) Sem nome | 0 | 0.0
- (D) Nenhuma saída, pois o construtor padrão gera erro
- (E) Sem nome | 17 | 8.5

Questão 13 (Aula 2 — Múltipla escolha)

Considere o construtor abaixo:

```
public Aluno(String nome, int idade, double nota) {  
    this.nome = nome;  
    this.idade = idade;  
    this.nota = nota;  
}
```

Ao executar "Aluno a2 = new Aluno("Maria", 17, 8.5);" seguido de "a2.exibir();", qual será a saída exibida?

- (A) Sem nome | 0 | 0.0
- (B) Erro de compilação

- (C) Maria | 0 | 0.0
- (D) Maria | 17 | 8.5
- (E) 17 | Maria | 8.5

Questão 14 (Aula 2 — Múltipla escolha)

O que a palavra-chave `this` representa dentro de um método ou construtor?

- (A) Uma nova instância de outra classe.
- (B) O nome da classe pai.
- (C) Um valor booleano padrão.
- (D) Um comando para encerrar o programa.
- (E) O objeto atual, ou seja, a própria instância sendo criada/manipulada.

Questão 15 (Aula 2 — Múltipla escolha)

Quando o uso da palavra-chave `this` é necessário para diferenciar atributo e parâmetro?

- (A) Quando o nome do parâmetro do construtor/método é igual ao nome do atributo da classe.
- (B) Nunca é necessário, o `this` é apenas estético.
- (C) Apenas quando a classe tem mais de 10 atributos.
- (D) Apenas em métodos estáticos.
- (E) Apenas dentro do método `main`.

Questão 16 (Aula 2 — Associação de colunas)

Associe a coluna A à coluna B, escrevendo o número correspondente entre parênteses:

Coluna A — Conceito	Coluna B — Definição
1. Construtor	() Referencia o objeto atual, diferenciando parâmetro de atributo
2. <code>this</code>	() Mesma classe com múltiplos construtores de assinaturas diferentes
3. Sobrecarga de construtor	() Método especial chamado automaticamente ao criar um objeto com <code>new</code>

Questão 17 (Aula 2 — Verdadeiro ou Falso)

Sobre `this` e sobrecarga de construtor, avalie as afirmações:

- () I. `this.nome` refere-se ao atributo `nome` do objeto atual, enquanto `nome` (sem `this`) pode se referir ao parâmetro recebido.
- () II. Sobrecarga de construtor significa que uma classe só pode ter um único construtor.
- () III. Dois construtores com o mesmo nome de classe, mas com parâmetros diferentes, são um exemplo de sobrecarga.
- () IV. `this` só pode ser usado dentro de construtores, nunca dentro de outros métodos.
- () V. A sobrecarga de construtor permite criar objetos de formas diferentes, dependendo dos dados disponíveis no momento.

Questão 18 (Aula 2 — Análise de afirmações)

- I. `this.nome` refere-se ao atributo `nome` do objeto atual.
- II. `nome` (sem `this`), dentro do construtor com parâmetros, refere-se ao parâmetro recebido.
- III. `this.nome` e `nome` sempre apontam para exatamente a mesma variável, sem nenhuma distinção.

Está(ão) correta(s):

- (A) Apenas III
- (B) Apenas I e II
- (C) I, II e III
- (D) Apenas I

(E) Apenas II e III

Questão 19 (Aula 2 — Múltipla escolha)

Por que é possível ter tanto "new Aluno()" quanto "new Aluno("Maria", 17, 8.5)" funcionando na mesma classe Aluno?

- (A) Porque o Java ignora os parâmetros extras automaticamente.
- (B) Porque "new Aluno()" sempre gera erro; apenas o segundo funciona de fato.
- (C) Porque a classe possui dois construtores com assinaturas diferentes — sobrecarga de construtor.
- (D) Porque atributos em Java não precisam de construtores.
- (E) Porque o Java converte automaticamente um construtor no outro.

Questão 20 (Aula 2 — Complete com o banco de palavras)

Banco de palavras: classe | retorno | this

"Um construtor tem o mesmo nome da _____ e não possui tipo de _____. A palavra-chave _____ referencia o objeto atual."

Questão 21 (Aula 2 — Múltipla escolha)

Observando a classe Aluno (com atributos nome, idade e nota, e um método exibir() que imprime esses valores), qual alternativa descreve corretamente o construtor padrão (sem parâmetros)?

- (A) Ele recebe nome, idade e nota como parâmetros e os atribui usando this.
- (B) Ele chama automaticamente o método exibir().
- (C) Ele não inicializa nenhum atributo, deixando todos como null.
- (D) Ele inicializa nome com "Sem nome", idade com 0 e nota com 0.0, sem receber parâmetros.
- (E) Ele é opcional e nunca pode ser definido manualmente.

Questão 22 (Aula 2 — Múltipla escolha)

O que caracteriza a sobrecarga de construtor (constructor overloading)?

- (A) Vários métodos com o mesmo nome em classes diferentes.
- (B) Um erro comum que deve ser evitado ao programar em Java.
- (C) Um único construtor que aceita qualquer quantidade de parâmetros automaticamente.
- (D) A substituição total do construtor padrão por outro.
- (E) Múltiplos construtores na mesma classe, cada um com uma lista diferente de parâmetros (assinatura).

Questão 23 (Aula 2 — Múltipla escolha)

Qual é a principal utilidade de se ter múltiplos construtores (sobrecarga) em uma classe?

- (A) Permitir criar objetos de formas diferentes, dependendo das informações disponíveis no momento da criação.
- (B) Deixar o código mais lento de propósito.
- (C) Impedir que o objeto seja criado sem parâmetros.
- (D) Aumentar o número de atributos da classe automaticamente.
- (E) Não há nenhuma utilidade prática.